

Dispatch Control for Priority-Based, Multi-Tenant Workflows under External Rate Limits

Stanislav Kosorin, Matr.-Nr. 457294

March 11, 2026

Workflow engines coordinate long-running distributed applications through durable execution, task queues, and automatic retries. In many production systems, these workflows primarily orchestrate calls to external APIs. While tasks may include local computation, contention is often dominated by access to externally rate-limited services. Such providers enforce fixed concurrency limits and rate budgets that, although documented and known to operators, are enforced outside the workflow engine. Workflow engines typically do not model this capacity explicitly. Instead, they dispatch tasks based on queue state, worker availability, and static priorities. Provider overload therefore becomes visible only after a request has been rejected, for example through a throttling response.

Workflow engines treat throttling as task failure and apply the configured retry policy. Retried tasks re-enter the system with the same priority as the original attempt, and the scheduler does not distinguish them from first attempts. Dispatch behavior therefore remains unchanged during provider saturation. This creates a problematic feedback loop in which retries consume dispatch capacity without increasing successful request rates. High-priority tasks may retry repeatedly before succeeding, while lower-priority tasks experience increased waiting times even when they themselves would not trigger throttling. Task backlogs can consequently grow during saturation periods, even when average arrival rates remain below documented provider limits.

These dynamics are amplified in multi-tenant deployments. A single workflow cluster commonly serves multiple tenants who share infrastructure and external providers but differ in workload mix and priority structure. Isolation typically relies on static priorities or queue separation. Under provider throttling, however, retries from one tenant can occupy dispatch capacity and delay work from other tenants, introducing cross-tenant interference. The interaction between fixed priorities, automatic retries, shared provider limits, and tenant concurrency ultimately determines tail latency, fairness, and backlog behavior in ways that current dispatch mechanisms do not address.

Related Work

Recent research on workflow execution systems has addressed durability, efficiency, and scalability. ExoFlow [1] decouples execution from recovery, enabling configurable tradeoffs between performance and fault tolerance while maintaining exactly-once semantics. Netherite [2] improves serverless workflow throughput through group commit and persistence pipelining, demonstrating that causally dependent workflow steps can commit together efficiently. Boki [3] introduces the metalog mechanism to address log ordering, read consistency, and fault tolerance in stateful serverless applications, achieving up to 4.7x speedups for workflow execution.

Admission control and overload management have a long history in systems research. Welsh et al. [4] introduce the staged event-driven architecture, decomposing applications into stages connected by explicit queues with per-stage admission control and dynamic resource controllers. When queues exceed thresholds, SEDA applies adaptive load shedding. Breakwater [5] addresses overload control for microsecond-scale RPCs through server-driven admission control that issues credits based on queueing delay, demonstrating that reactive schemes can converge to stable performance within 20ms while earlier approaches like SEDA require over one second to recover from congestion collapse.

Distributed rate limiting has been studied extensively in networking contexts. Raghavan et al. [6] introduce two algorithms for enforcing global rate limits across distributed limiters: Global Random Drop, which calculates drop probability from excess demand using gossip protocols, and

Flow Proportional Share, which dynamically adjusts local token bucket limits proportional to weighted flow counts. The latter proves more robust to network delays because flow counts are more stable than instantaneous throughput. Seawall [7] extends this to multi-tenant data centers, implementing link-oriented congestion control with per-tenant weights that allocates bandwidth according to weighted fair share, preventing any tenant from monopolizing shared network capacity.

Multi-tenant fairness in shared infrastructure has also received attention. Sheng et al. [8] propose the Virtual Token Counter algorithm for fair scheduling in LLM serving, providing a provable bound on service difference between backlogged clients while remaining work-conserving. Shockwave [9] extends market theory to dynamic settings, co-optimizing efficiency and fairness for workloads with time-varying resource requirements.

These systems advance workflow execution, admission control, distributed rate limiting, and multi-tenant fairness independently. However, none address dispatch control under external rate limits in workflow engines. Existing admission control schemes assume the system controls the constrained resource and can directly observe queue depths. Distributed rate limiters assume the enforcing system controls the limited resource. External API rate limits differ in that capacity is enforced remotely, throttling responses arrive asynchronously after requests have already consumed dispatch capacity, and the workflow engine cannot directly observe provider queue state. This thesis addresses this gap by introducing rate-limit-aware dispatch that operates on top of existing workflow engines without modifying their internals, using throttling feedback to coordinate dispatch decisions.

Approach

This thesis compares provider-agnostic dispatch with dispatch that accounts for provider limits and observed throttling, under fixed priorities and retry semantics. It introduces a dispatch control layer that operates on top of a workflow engine without modifying its internals. The workflow engine remains responsible for workflow execution, state management, and retry handling. The control layer decides when tasks are released into task queues. It maintains counters for provider concurrency and rate limits and observes request outcomes, including throttling responses. Provider limits are treated as static over the considered timescales. The scheduler may delay or release tasks. It does not drop tasks, change priorities, reorder tasks across priority classes, or modify retry behavior.

This design involves fundamental tradeoffs that cannot be fully mitigated and must be evaluated empirically. First, distributed counter coordination requires choosing between strong consistency, which adds latency to every dispatch decision and limits throughput, and eventual consistency, which reduces latency but permits temporary overshoot of provider limits during bursts. Second, the control layer must balance reactivity against stability: aggressive adaptation to throttling signals risks oscillation between over- and under-utilization, while conservative adaptation accumulates backlogs before responding to saturation. Third, work-conserving dispatch that uses spare capacity for lower-priority tasks maximizes throughput but risks priority inversion: if high-priority work arrives after low-priority work has consumed the provider’s rate budget, high-priority tasks must wait for the next quota window. Reserving capacity for high-priority work avoids this inversion but wastes throughput when high-priority demand is lower than its allocation.

The evaluation quantifies these tradeoffs using replayed production workflows and controlled synthetic workloads, with Temporal as the underlying workflow engine. Tenants share provider capacity while generating independent workloads. Synthetic workloads vary arrival rates, priority mixes, and burst patterns to isolate individual tradeoffs under controlled conditions. Production traces capture realistic correlations between tenants and temporal patterns that synthetic workloads may miss.

Each tradeoff maps to specific metrics. The consistency-latency tradeoff surfaces in dispatch decision latency and limit overshoot rates during load spikes. The reactivity-stability tradeoff manifests as convergence time after saturation onset and backlog growth during transient overload. The priority-utilization tradeoff appears in priority inversion frequency and reserved capacity waste when high-priority demand falls below allocation. Beyond tradeoff-specific metrics, tail latency per priority and per tenant, retry rates, and worst-case waiting times characterize end-to-end system behavior. The expectation is that provider-agnostic priority dispatch shows elevated retry rates, cross-tenant interference, and persistent backlog growth under provider saturation, while rate-limit-aware dispatch reduces retries, limits backlog growth, and lowers tail latency without violating fixed priority semantics.

References

- [1] Siyuan Zhuang, Stephanie Wang, Eric Liang, Yi Cheng, and Ion Stoica. ExoFlow: A Universal Workflow System for Exactly-Once DAGs. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 269–286, 2023.
- [2] Sebastian Burckhardt, Badrish Chandramouli, Chris Gillum, David Justo, Konstantinos Kallas, Connor McMahon, Christopher S. Meiklejohn, and Xiangfeng Zhu. Netherite: Efficient Execution of Serverless Workflows. *Proc. VLDB Endow.*, 15(8):1591–1604, 2022.
- [3] Zhipeng Jia and Emmett Witchel. Boki: Stateful Serverless Computing with Shared Logs. In *ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP ’21)*, pages 691–707, 2021.
- [4] Matt Welsh, David Culler, and Eric Brewer. SEDA: An Architecture for Well-Conditioned, Scalable Internet Services. In *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP ’01)*, pages 230–243. ACM, 2001.
- [5] Inho Cho, Ahmed Saeed, Joshua Fried, Seo Jin Park, Mohammad Alizadeh, and Adam Belay. Overload Control for μ s-scale RPCs with Breakwater. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 299–314. USENIX Association, 2020.
- [6] Barath Raghavan, Kashi Vishwanath, Sriram Ramabhadran, Kenneth Yocum, and Alex C. Snoeren. Cloud Control with Distributed Rate Limiting. In *Proceedings of the 2007 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications (SIGCOMM ’07)*, pages 337–348. ACM, 2007.
- [7] Alan Shieh, Srikanth Kandula, Albert Greenberg, Changhoon Kim, and Bikas Saha. Sharing the Data Center Network. In *8th USENIX Symposium on Networked Systems Design and Implementation (NSDI 11)*, pages 309–322. USENIX Association, 2011.
- [8] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E. Gonzalez, and Ion Stoica. Fairness in Serving Large Language Models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 965–988, 2024.
- [9] Pengfei Zheng, Rui Pan, Tarannum Khan, Shivaram Venkataraman, and Aditya Akella. Shockwave: Fair and Efficient Cluster Scheduling for Dynamic Adaptation in Machine Learning. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 703–723, 2023.